

Practical 1 PDF Submissions

Task 1: Source Connectors

1.5 Explain why Factory Method is better here than one `createConnector(string kind)` function with an if/else chain. Refer to the open-closed principle.

The Factory Method adheres to the Open-Closed Principle because adding a new connector (e.g., a `MongoDbConnector`) only requires adding a new `MongoDbFactory` subclass. The existing codebase remains completely untouched. A single `createConnector(string kind)` function with an if/else chain forces you to open and modify existing code every time a new connector is onboarded, violating the Open-Closed Principle.

1.6 State which Factory Method participant each of `Connector`, `CsvConnector`, `ConnectorFactory`, `CsvFactory` plays.

- `Connector`: Product
- `CsvConnector`: ConcreteProduct
- `ConnectorFactory`: Creator
- `CsvFactory`: ConcreteCreator

Task 2: Transformations

2.5 Explain shallow versus deep copy, state which `clone()` must produce, and why the clone must be independent of the prototype.

A shallow copy only copies memory addresses (pointers), meaning the new object points to the exact same underlying memory as the original. A deep copy allocates brand new memory and copies the actual values, creating a fully separate object. `clone()` must produce a deep copy. The clone must be independent so that if a pipeline modifies or deletes the clone during its run, the original prototype stored in the registry remains safe and intact for future cloning.

2.6 Explain why the registry returns `clone()`s rather than the stored pointer, and what breaks if two pipelines shared one prototype.

The registry returns clones to ensure each pipeline gets its own independent instance of the transformation. If the registry returned the stored pointer, multiple pipelines would share the exact same prototype object in memory. If one pipeline modified the transformation's internal state or deleted it when the pipeline finished, it would corrupt or instantly crash the other pipelines relying on that shared object.

Task 3: The pipeline lifecycle

3.5 Explain why `run()` is deliberately not virtual, and what a subclass could break if it could override it.

`run()` is deliberately not virtual because it is the Template Method that defines the strict, invariant skeleton of the algorithm (`connect -> extract -> transform -> load`). If a subclass could override it, the subclass could maliciously or accidentally change the fundamental order of execution (e.g., loading before extracting) or skip critical steps entirely, completely breaking the core pipeline lifecycle.

3.6 Classify each Pipeline method. State what the Factory Method and Template Method each contribute inside Pipeline and why they do not overlap.

- Template method: `run()`
- Concrete steps: `connect()`, `transform()`
- Primitive operations: `extract()`, `load()`

Inside the Pipeline, the Factory Method is responsible strictly for creation (spawning the correct Connector object without hardcoding its concrete type). The Template Method is responsible strictly for behavioral control flow (dictating the exact invariant sequence in which `extract`, `transform`, and `load` are called). They do not overlap because the Factory Method handles

how things are built, while the Template Method handles when they are executed.

Task 4: Checkpointing and recovery

4.4 Explain the narrow and wide interfaces of a Memento, and which participant may use each. The narrow interface provides limited access (only allowing the object to be passed around and stored safely) and is used by the Caretaker (CheckpointManager). The wide interface allows full access to read and write the internal state data (e.g., `getStage()`, `getRecords()`) and is exclusively used by the Originator (Pipeline) to capture and restore its own state.

4.5 Describe step by step how a run that fails during load recovers and resumes.

- 1. During a successful transform phase, the Pipeline calls `createCheckpoint()` to create a `RunCheckpoint` holding the current stage and records.
- 2. The engine tells the CheckpointManager to call `save()`, which stores this checkpoint in its history.
- 3. The run proceeds to the load stage, but fails and the system crashes.
- 4. Upon recovery, the engine asks the CheckpointManager to call `undo()`, which removes and returns the last saved `RunCheckpoint`.
- 5. The engine passes this checkpoint back to the Pipeline via `restore()`, which resets its internal stage and records to the exact state they were in right before load failed, allowing the pipeline to safely resume.

4.6 State which Memento participant each of Pipeline, `RunCheckpoint`, `CheckpointManager` plays.

- Pipeline: Originator
- `RunCheckpoint`: Memento
- CheckpointManager: Caretaker